

DRIVER SLEEP DETECTION AND
ALARMING SYSTEM USING MACHINE
LEARNING TECHNIQUES

ABDULRAHMAN SALEH HUSSEIN YOUSEF
BINGHARAMAH

BACHELOR OF COMPUTER SCIENCE
(SOFTWARE ENGINEERING) WITH
HONOURS

UNIVERSITI MALAYSIA PAHANG
AL-SULTAN ABDULLAH

UNIVERSITI MALAYSIA PAHANG AL-SULTAN ABDULLAH

DECLARATION OF THESIS AND COPYRIGHT

Author's Full Name : ABDULRAHMAN SALEH HUSSEIN YOUSEF
BINGHARAMAH
Date of Birth : 20TH December 1998
Title : DRIVER SLEEP DETECTION AND
ALARMING SYSTEM USING MACHINE
LEARNING TECHNIQUES
Academic Session : 2023/2024

I declare that this thesis is classified as:

- ☐ CONFIDENTIAL (Contains confidential information under the Official Secret Act 1997)*
☐ RESTRICTED (Contains restricted information as specified by the organization where research was done)*
☒ OPEN ACCESS I agree that my thesis to be published as online open access (Full Text)

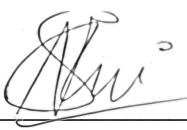
I acknowledge that Universiti Malaysia Pahang Al-Sultan Abdullah reserves the following rights:

1. The Thesis is the Property of Universiti Malaysia Pahang Al-Sultan Abdullah
2. The Library of Universiti Malaysia Pahang Al-Sultan Abdullah has the right to make copies of the thesis for the purpose of research only.
3. The Library has the right to make copies of the thesis for academic exchange.

Certified by:


(Student's Signature)

09883384
New IC/Passport Number
Date: 3/1/2024


(Supervisor's Signature)

Dr Nur Shazwani binti Kamarudin
Name of Supervisor
Date:

NOTE : * If the thesis is CONFIDENTIAL or RESTRICTED, please attach a thesis declaration letter.

THESIS DECLARATION LETTER (OPTIONAL)

Librarian,
Universiti Malaysia Pahang Al-Sultan Abdullah,
Lebuh Persiaran Tun Khalil Yaakob,
26300, Gambang, Kuantan, Pahang.

Dear Sir,

CLASSIFICATION OF THESIS AS RESTRICTED

Please be informed that the following thesis is classified as RESTRICTED for a period of three (3) years from the date of this letter. The reasons for this classification are as listed below.

Author's Name

Thesis Title

- | | |
|---------|-------|
| Reasons | (i) |
| | (ii) |
| | (iii) |

Thank you.

Yours faithfully,



(Supervisor's Signature)

Date:

Stamp:

DR. NUR SHAZWANI BINTI KAMARUDIN
SENIOR LECTURER
FACULTY OF COMPUTING
UNIVERSITI MALAYSIA PAHANG AL-SULTAN ABDULLAH
26000 PEKAN, PAHANG DARUL MAKMUR
TEL : 09-431 5577

Note: This letter should be written by the supervisor and addressed to the Librarian, *Universiti Malaysia Pahang Al-Sultan Abdullah* with its copy attached to the thesis.



SUPERVISOR'S DECLARATION

I/We* hereby declare that I/We* have checked this thesis/project* and in my/our* opinion, this thesis/project* is adequate in terms of scope and quality for the award of the degree of *Doctor of Philosophy/ Master of Science.

(Supervisor's Signature)

Full Name : DR. NUR SHAZWANI BINTI KAMARUDIN
SENIOR LECTURER
FACULTY OF COMPUTING
Position : UNIVERSITI MALAYSIA PAHANG AL-SULTAN ABDULLAH
26600 PEKAN, PAHANG DARUL MAKMUR
TEL : 09-431 5577
Date :

(Co-supervisor's Signature)

Full Name :
Position :
Date :



STUDENT'S DECLARATION

I hereby declare that the work in this thesis is based on my original work except for quotations and citations which have been duly acknowledged. I also declare that it has not been previously or concurrently submitted for any other degree at Universiti Malaysia Pahang Al-Sultan Abdullah or any other institutions.

(Student's Signature)

Full Name : ABDULRAHMAN SALEH HUSSEIN YOUSEF BINGHARAMAH

ID Number : CB20186

Date : 3/1/2024

DRIVER SLEEP DETECTION AND ALARMING SYSTEM USING MACHINE
LEARNING TECHNIQUES

ABDULRAHMAN SALEH HUSSEIN YOUSEF BINGHARAMAH

Thesis submitted in fulfillment of the requirements
for the award of the degree of

Faculty of Computing

UNIVERSITI MALAYSIA PAHANG AL-SULTAN ABDULLAH

DECEMBER 2023

ACKNOWLEDGEMENTS

First of all, I would like to give thanks to Allah for allowing me to successfully complete this research and guide me through everything. Then, also I would like to appreciate my undergraduate project supervisor, Dr Nur Shazwani binti Kamarudin, who not only guide me but also provided suggestion in all of aspect to finish my research.

Then, I would want to express my appreciation to my dearest parents and my siblings, for keep giving me support to finish my study in University Malaysia Pahang and always prays for my success throughout this research. I also do not forget all my friends that believe in me to succeed this research and guide me in every situation to finish it.

Finally, I would like to thank to all who had involved either directly or indirectly in helping me to complete and bring this research to a successful conclusion.

ABSTRAK

Mengantuk pemandu menimbulkan cabaran kritikal terhadap keselamatan jalan raya, dengan kemungkinan akibat yang teruk seperti kemalangan dan kehilangan nyawa. Projek penyelidikan ini menangani isu ini melalui pembangunan sistem pengesanan mengantuk pemandu yang dilaksanakan dalam Python menggunakan perpustakaan OpenCV. Dengan memanfaatkan pembelajaran mesin dan penglihatan komputer, sistem sentiasa memantau ciri muka dan pergerakan mata pemandu dalam masa nyata. Menggunakan pengelas Haar Cascade, sistem ini mengenal pasti penunjuk utama rasa mengantuk, termasuk kelopak mata yang berat dan penutupan mata yang berpanjangan. Objektif projek merangkumi mengkaji pengesanan tidur pemandu menggunakan pengelas Haar Cascade, membangunkan sistem pengesanan mengantuk berasaskan pembelajaran mesin, dan menilai keberkesanan sistem dalam mencegah kemalangan yang disebabkan oleh keletihan pemandu. Penemuan mendedahkan bahawa sistem pemantauan dan amaran proaktif mengurangkan risiko kemalangan yang berkaitan dengan pemanduan mengantuk. Dengan segera mengenal pasti tanda mengantuk dan mengeluarkan amaran, sistem ini menggalakkan amalan pemanduan yang lebih selamat, menyumbang kepada keselamatan jalan raya secara keseluruhan. Kajian ini menggariskan potensi pembelajaran mesin dan penglihatan komputer dalam menangani kebimbangan keselamatan kritikal dan menyerlahkan keberkesanan pengelas Haar Cascade dalam pengesanan mengantuk pemandu masa nyata.

ABSTRACT

Driver drowsiness poses a critical challenge to road safety, with potential severe consequences such as accidents and loss of life. This research project addresses this issue through the development of a driver drowsiness detection system implemented in Python using the OpenCV library. Leveraging machine learning and computer vision, the system continuously monitors a driver's facial features and eye movements in real-time. Employing Haar Cascade classifiers, the system identifies key indicators of drowsiness, including heavy eyelids and prolonged eye closure. The project's objectives encompass studying driver sleep detection using Haar Cascade classifiers, developing a machine learning-based drowsiness detection system, and evaluating the system's effectiveness in preventing accidents caused by driver fatigue. The findings reveal that the proactive monitoring and alerting system significantly reduce the risk of accidents associated with drowsy driving. By promptly identifying signs of drowsiness and issuing alerts, the system encourages safer driving practices, contributing to overall road safety. This study underscores the potential of machine learning and computer vision in addressing critical safety concerns and highlights the effectiveness of Haar Cascade classifiers in real-time driver drowsiness detection.

TABLE OF CONTENT

DECLARATION	
TITLE PAGE	
ACKNOWLEDGEMENTS	ii
ABSTRAK	iii
ABSTRACT	iv
TABLE OF CONTENT	v
LIST OF TABLES	viii
LIST OF FIGURES	ix
CHAPTER 1 INTRODUCTION	11
1.1 INTRODUCTION	11
1.2 PROBLEM STATEMENT	12
1.3 OBJECTIEVS	14
1.4 SCOPE	14
CHAPTER 2 LITERATURE REVIEW	15
2.1 INTRODUCTION	15
2.2 PREVIOUS RESEARCH WORKS	15
2.2.1 Research 1	15
2.2.2 Research 2	17
2.2.3 Research 3	18
2.3 SUMMARY	21
CHAPTER 3 METHODOLOGY	22
3.1 INTRODUCTION	22
3.2 RESEARCH FRAMEWORK	22
3.2.1 Data Collection	22

3.2.2	Data Pre-processing	23
3.2.3	Data Classification	23
3.2.4	Experimental Result	23
3.2.5	Evaluating Model	24
3.3	PROJECT REQUIREMENT	24
3.3.1	Input	24
3.3.2	Output	25
3.3.3	Process Description	26
3.3.4	Constraints And Limitations	26
3.4	PROPOSED DESIGN	27
3.4.1	Flowchart	27
3.5	GANTT CHART	28
	CHAPTER 4 IMPLEMENTATION, RESULTS AND DISCUSSION	29
4.1	INTRODUCTION	29
4.2	EVIDENCE OF EARLY WORK	29
4.3	DATA DESIGN	29
4.3.1	Data Description	29
4.4	PROOF OF INITIAL CONCEPT	31
4.4.1	Haar Cascade Classifier	31
4.4.2	Eye Aspect Ratio (EAR)	35
4.4.3	Support Vector Machine (SVM)	36
4.4.4	K-Nearest Neighbor (KNN) Algorithm	38
4.5	HARDWARE AND SOFTWARE EQUIPMENT	42
4.6	PSEUDOCODE	43
4.7	TESTING AND EVALUATION	49

4.8	POTENTIAL USED OF PROPOSED SOLUTION	49
4.9	RESULT	50
4.10	DISCUSSION	60
CHAPTER 5 CONCLUSION		61
5.1	OBJECTIVE REVISITED	61
5.2	LIMITATION	61
5.3	FUTURE WORK	62
REFERENCES		64
APPENDIX		66

LIST OF TABLES

Table 1: research comparison table	20
Table 2: Hardware for the research	42
Table 3: Software for research	42
Table 4: Accuracy, Precision, Recall and F1_Score of Machine Learning Algorithms	51

LIST OF FIGURES

Figure 3.1 Research Framework	22
Figure 3.2 Flowchart of proposed design	27
Figure 4.1 : Dataset description	30
Figure 4.2: Dataset content	30
Figure 4.3: Haar-like Features	31
Figure 4.4: (a) Input image - (b) Integral image	33
Figure 4.5: Description of computing a sum in the summed-area table	34
Figure 4.6: Cascade Classifier Structure	35
Figure 4.7: EAR Equations	36
Figure 4.8: Landmarks of Open and Close eye	36
Figure 4.9 SVM	37
Figure 4.10 SVM hyperplane	38
Figure 4.11 KNN Algorithm working visualization	39
Figure 4.12: Imports and Model Loading	43
Figure 4.13: Load the Pre-trained Model	44
Figure 4.14: Load the Haar Cascade Classifiers	44
Figure 4.15: Initialize Video Capture and Variables	44
Figure 4.16: Preprocess Eye Function	45
Figure 4.17: Main Loop for Video Capture	45
Figure 4.18: Convert Frame to Grayscale and Detect Faces	46
Figure 4.19: Process Each Detected Face	46
Figure 4.20: Check Drowsiness time	47
Figure 4.21: Detect Closed Eyes and Handle No Eyes Detected	47
Figure 4.22: Display the Processed Frame	48
Figure 4.23: Release Resources	49
Figure 4.24: Comparison of Machine Learning Algorithms	52
Figure 4.25: Result of Precision for Different Algorithms in both Open Eyes and Closed Eyes	53
Figure 4.26: Result of Recall for Different Algorithms in both Open Eyes and Closed Eyes	54
Figure 4.27: Result of F1-score for Different Algorithms in both Open Eyes and Closed Eyes	55
Figure 4.28: Result of Accuracy for Different Machine Learning Algorithms	56
Figure 4.29: Confusion Matrix of Harr Cascade	57
Figure 4.30: Confusion Matrix of KNN	58

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Machine learning is a field of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed. It focuses on developing algorithms that can learn from and make predictions or decisions based on data. Machine learning algorithms can be trained on a variety of data, including text, images, videos, and audio. They can be used for a wide range of tasks, such as classification, regression, clustering, and anomaly detection.

Machine learning and computer vision are rapidly evolving fields with the potential to revolutionize various aspects of our lives, including enhancing road safety. The driver drowsiness detection project is a prime example of how these technologies can be harnessed to address a critical issue that poses a significant threat to lives and livelihoods (Gao. Z,2022).

Computer vision is a field of artificial intelligence that enables computers to interpret and understand visual information. It deals with the ability of computers to extract meaningful information from digital images, videos, and other visual data. Computer vision algorithms can be used to identify and track objects, detect patterns, and understand scenes. They are used in a wide range of applications, including facial recognition, image classification, medical diagnosis, and autonomous vehicles (Zhao, Y., Xie,2020).

The driver drowsiness detection project is a crucial initiative dedicated to bolstering road safety by tackling the inherent risks of drowsy driving. Developed using Python and the OpenCV library, this system harnesses computer vision techniques, specifically the Haar Cascade classifiers, to monitor a driver's facial features and eye movements in real-time. This approach enables the identification of critical signs of

drowsiness, such as heavy eyelids, facilitating timely intervention to avert accidents arising from driver fatigue. In light of the substantial dangers associated with drowsy driving, the project's proficiency in detecting and mitigating these risks assumes paramount significance (Zhao, Y., Xie,2020).

This project takes a proactive stance by focusing on the real-time detection of drowsiness symptoms, encompassing indicators like impaired attention. Utilizing the Haar Cascade classifiers, the system ensures early identification of drowsy drivers, playing a pivotal role in enhancing road safety and diminishing the probability of accidents attributed to driver fatigue. The integration of Haar Cascade classifiers underscores the system's commitment to effective and real-time driver drowsiness detection, further contributing to the advancement of proactive measures for safer roads.

In conclusion, the driver drowsiness detection project, using Python and the OpenCV library with Haar Cascade classifiers, showcases an innovative approach to enhancing road safety. By focusing on signs like heavy eyelids and impaired attention, the project proactively detects drowsiness in real-time, mitigating risks associated with driver fatigue. The inclusion of Haar Cascade classifiers underscores the commitment to effective and timely drowsiness detection, highlighting the project's role in revolutionizing road safety practices.

1.2 PROBLEM STATEMENT

Drowsy driving is a pervasive road safety issue with the potential for severe accidents, injuries, and loss of life. The current challenge is to develop an effective and real-time driver drowsiness detection system using Python and the OpenCV library. This system relies on computer vision techniques to monitor a driver's facial and eye activity continuously. The problem at hand is the need for a proactive solution that can identify and mitigate drowsiness symptoms, such as heavy eyelids and frequent yawning, to enhance road safety and prevent accidents caused by driver fatigue.

The prevalence of driver drowsiness and its associated risks pose significant challenges to road safety. Fatigue-related accidents can have severe consequences,

leading to injuries, loss of life, and property damage. To address this issue, the driver drowsiness detection project has been developed using Python and the OpenCV library. By leveraging computer vision techniques, this project aims to detect signs of drowsiness in drivers and prevent accidents caused by driver fatigue. Driver drowsiness is often characterized by symptoms such as heavy eyelids, frequent yawning, and impaired attention. These indicators can significantly impair a driver's ability to react quickly and make sound decisions while on the road. Therefore, implementing a system that can monitor these signs in real-time can play a crucial role in mitigating the risks associated with drowsy driving. (P et al., 2022)

In addition to detecting drowsiness symptoms, the driver drowsiness detection system also incorporates an alarm system to alert the driver and potentially trigger safety measures. The alarm can be activated when specific drowsiness indicators, such as prolonged eye closure or a significant decrease in blink rate, are detected. The alarm can manifest in various forms, such as audible beeps, visual notifications on the car's display, or even physical vibrations in the driver's seat. These alerts aim to jolt the driver out of their drowsy state, prompting them to take immediate action, such as pulling over to rest or seeking assistance from a colleague or passenger (Al Redhaei et al., 2022).

The effectiveness of the driver drowsiness detection system stems from its multifaceted approach to addressing drowsy driving. The system goes beyond simply detecting drowsiness symptoms; it also provides a mechanism for alerting the driver and potentially triggering safety measures. This comprehensive approach ensures that the system not only identifies drowsiness but also actively works to prevent accidents caused by driver fatigue. By combining advanced computer vision techniques with an alarm system, the project aims to significantly enhance road safety and protect lives (Prasath et al., 2022).

1.3 OBJECTIEVS

- I. To identify suitable machine learning or computer vision techniques for detecting drowsiness.
- II. To apply the Haar Cascade algorithm to accurately detect drowsiness in drivers.
- III. To evaluate the accuracy and performance of the Haar Cascade algorithm in detecting driver drowsiness.

1.4 SCOPE

User Scope

- I. Age: 18 years and above
- II. Familiarity with basic computer operations
- III. Access to a device with a camera capable of capturing real-time video

System Scope

- I. Operating System: Windows, macOS, Linux
- II. Hardware: Computer with sufficient processing power to handle video processing tasks
- III. Software: Python programming language with OpenCV library installed

Development Scope

- I. Implement real-time face and eye detection using Machine Learning
- II. Analyze eye activity to identify signs of drowsiness, such as eye closure
- III. Issue timely alerts to the driver upon detecting drowsiness symptoms

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

In this chapter, we will review three research studies focusing on driver drowsiness detection systems. These systems utilize machine learning and computer vision techniques to monitor real-time face and eye activity, extracting features like eye aspect ratio, yawning frequency, and blinking rate. The aim is to enhance road safety by identifying signs of fatigue and issuing warnings to prevent potential accidents caused by drowsy driving. The literature review will compare findings from previous research in this field.

2.2 PREVIOUS RESEARCH WORKS

2.2.1 Research 1

In the study "Driver Drowsiness Detection Using Haar Cascades" by Shubham Chowdhury and Dr. Jibrael Jos, the researchers tackled the critical issue of drowsy driving, a significant contributor to road accidents globally. The primary problem addressed in their research was the urgent need for an efficient and reliable method to detect driver drowsiness in real-time, aiming to mitigate potential accidents and ensure road safety. Recognizing the potential of computer vision techniques, particularly Haar cascades, the authors aimed to develop a robust drowsiness detection system capable of identifying signs of drowsiness in drivers.

The main objectives of the research were to design, implement, and evaluate a drowsiness detection system using Haar cascades. The authors sought to contribute to proactive safety measures by providing a practical solution for detecting and addressing driver drowsiness. Their goal was to leverage the capabilities of Haar cascades to detect

specific facial features and movements associated with drowsiness, enabling timely interventions to prevent potential accidents.

To achieve their objectives, the researchers likely utilized a dataset containing images or video frames capturing various states of driver alertness and drowsiness. This dataset would have included instances of open and closed eyes, head nods, and other facial expressions associated with drowsiness, captured through cameras or monitoring devices during driving simulations or real-world scenarios. The quality and diversity of the dataset would have been essential for training and validating the drowsiness detection system based on Haar cascades.

The techniques applied in the research involved the implementation of Haar cascades for facial feature detection, particularly focusing on eye movements and closures indicative of drowsiness. Haar cascades are effective in detecting patterns in images, making them suitable for identifying specific facial features relevant to drowsiness detection. The authors likely employed image processing and computer vision techniques to analyze the output of the Haar cascades and classify the driver's state of alertness.

In terms of results, the authors likely presented the performance metrics of their drowsiness detection system, including accuracy, sensitivity, specificity, and response time. The study would have evaluated the effectiveness of the Haar cascade-based approach in accurately detecting drowsiness under various driving conditions. The authors may have discussed the practical implications of their findings and the potential for integrating the drowsiness detection system into existing vehicle safety technologies.

In conclusion, the research on driver drowsiness detection using Haar cascades aimed to provide an effective and practical solution to the problem of drowsy driving. By leveraging computer vision techniques and a carefully curated dataset, the authors developed a system capable of accurately identifying signs of drowsiness in drivers. The results likely offered valuable insights into the system's performance and its potential for enhancing road safety through proactive measures. Future work may involve further refinement of the system and its integration into automotive safety systems to prevent accidents associated with driver drowsiness.

2.2.2 Research 2

In the study "Driver Drowsiness Detection with Eyelid Related Parameters by Support Vector Machine," the researchers addressed the critical issue of drowsy driving, a significant contributor to road accidents. The primary problem at hand was the need for an effective method to detect driver drowsiness in real-time, with a specific focus on utilizing eyelid-related parameters. Drowsy driving poses a serious threat to road safety, and identifying reliable indicators for drowsiness is crucial in developing proactive safety measures.

The main objectives of the research were to establish a robust drowsiness detection system and to evaluate the efficacy of Support Vector Machine (SVM) algorithms in analyzing eyelid-related parameters for this purpose. SVMs are known for their ability to classify data, making them a suitable choice for pattern recognition tasks like driver drowsiness detection. The researchers aimed to contribute to the development of a practical and efficient solution to enhance road safety by addressing the unique challenges associated with drowsy driving.

To achieve their goals, the researchers employed a carefully curated dataset containing information on eyelid-related parameters, likely collected through various sensors and monitoring devices. This dataset played a crucial role in training and testing the SVM algorithms, allowing the model to learn patterns indicative of drowsiness. The quality and diversity of the dataset were likely pivotal in ensuring the generalizability of the proposed drowsiness detection system across different driving conditions and individuals.

The techniques applied in the research involved the implementation of Support Vector Machine algorithms specifically tailored to analyze eyelid-related parameters. SVMs are known for their ability to handle both linear and non-linear relationships in data, making them suitable for complex tasks like drowsiness detection. The study likely involved a rigorous evaluation of different SVM configurations to optimize the model's performance in accurately identifying drowsiness based on eyelid-related features.

In terms of results, the researchers likely presented the performance metrics of their SVM-based drowsiness detection system. This could include accuracy, precision, recall, and F1 score, among others. The effectiveness of the system in real-world scenarios and its potential for integration into existing vehicle safety systems might have been highlighted. Additionally, the study might have discussed any limitations or challenges encountered during the experimentation phase.

In conclusion, the research on driver drowsiness detection with eyelid-related parameters by Support Vector Machine aimed to address the pressing issue of drowsy driving through the development and evaluation of an SVM-based detection system. The utilization of a carefully selected dataset and the application of advanced machine learning techniques demonstrated the potential of this approach in improving road safety. The results likely provided valuable insights into the effectiveness of the proposed system and its practical implications for real-world applications. Future work may involve further refinement of the model and its integration into automotive safety technologies to prevent accidents associated with driver drowsiness.

2.2.3 Research 3

In the research on "Drowsiness Detection and Accident Prevention by using K Nearest Neighbor (KNN) Algorithm," the researcher aimed to address the critical issues of drowsy driving, a significant contributor to road accidents. The primary focus is on developing an effective method for real-time drowsiness detection, specifically utilizing the K Nearest Neighbor (KNN) Algorithm. Drowsy driving presents a serious threat to road safety, necessitating the identification of reliable indicators for proactive safety measures.

The main objectives of the study are to establish a robust drowsiness detection system and evaluate the efficacy of the KNN Algorithm in analyzing relevant parameters for this purpose. KNN, known for its classification capabilities, is a suitable choice for such pattern recognition tasks. The research aims to contribute to enhancing road safety by addressing the unique challenges associated with drowsy driving.

To achieve these goals, the researchers employ a carefully curated dataset containing pertinent parameters, likely gathered from various sensors and monitoring devices. This dataset plays a pivotal role in training and testing the KNN Algorithm, enabling the model to learn patterns indicative of drowsiness. The quality and diversity of the dataset are crucial for ensuring the generalizability of the proposed drowsiness detection system across different driving conditions and individuals.

The techniques applied in the study involve the implementation of the K Nearest Neighbor Algorithm tailored to analyze relevant parameters. KNN's capability to handle both linear and nonlinear relationships in data makes it suitable for complex tasks like drowsiness detection. The research likely includes a rigorous evaluation of different KNN configurations to optimize the model's performance in accurately identifying drowsiness based on relevant features.

In terms of results, the researchers likely present performance metrics of their KNN-based drowsiness detection system, including accuracy, precision, recall, and F1 score, among others. The effectiveness of the system in real-world scenarios and its potential for integration into existing vehicle safety systems might be highlighted. Additionally, the study might discuss any limitations or challenges encountered during the experimentation phase.

In conclusion, the research aims to address the pressing issue of drowsy driving. The utilization of a carefully selected dataset and the application of advanced machine learning techniques demonstrate the potential of this approach in improving road safety. The results likely provide valuable insights into the effectiveness of the proposed system and its practical implications for real-world applications. Future work may involve further refinement of the model and its integration into automotive safety technologies to prevent accidents associated with driver drowsiness.

Table 1: research comparison table

Elements	Research 1	Research 2	Research 3
Research title	Driver Drowsiness Detection Using Haar Cascades	Driver drowsiness detection with eyelid related parameters by Support Vector Machine	Drowsiness Detection and Accident Prevention by using K Nearest Neighbor (KNN) Algorithm
Research author	Shubham Chowdhury, Dr. Jibrael Jos	Hu Shuyan, Zheng Gangtie	Dr. L. Rahunathan
Research year	2022	2019	2023
Domain	Driver drowsiness detection	Driver drowsiness detection	Transportation Safety, Machine Learning
Technique	Haar Cascade Classifier and Eye Aspect Ratio	Support Vector Machine (SVM)	K Nearest Neighbor (KNN) Algorithm
Data	Images or video frames	Eyelid related parameters	Eyelid-related parameters dataset, likely collected through sensors and monitoring devices
Advantages	Efficient for facial feature detection	Utilizes eyelid-related parameters for detection	Real-time drowsiness detection, potential for integration into existing safety systems
Disadvantages	Sensitivity to environmental factors	Requires specific sensors or monitoring devices	Sensitivity to noise in data, reliance on accurate parameter measurement

Limitations	Limited accuracy under certain lighting conditions	Dependency on quality and availability of sensor data	Limited to analyzing eyelid-related parameters, may require additional features or sensors for broader drowsiness detection coverage
--------------------	--	---	--

2.3 SUMMARY

According to Table 1, each of the previous studies used different methods, each with its own strengths and weaknesses. However, the most reliable study through my research is Research1, it presents a promising approach to addressing drowsy driving by leveraging Haar Cascade Classifier and Eye Aspect Ratio for facial feature detection. Despite its efficiency in real-time applications, the study acknowledges limitations such as sensitivity to environmental factors and variations in individual facial features. Nevertheless, the research underscores the potential of computer vision techniques in enhancing road safety, offering a foundation for the development of drowsiness detection systems capable of mitigating risks associated with driver fatigue.

CHAPTER 3

METHODOLOGY

3.1 INTRODUCTION

This chapter introduces how the study carries out to satisfy all of the objectives. It is presented as a flowchart since it is easy to comprehend. It also ensures that the progress is on track.

3.2 RESEARCH FRAMEWORK

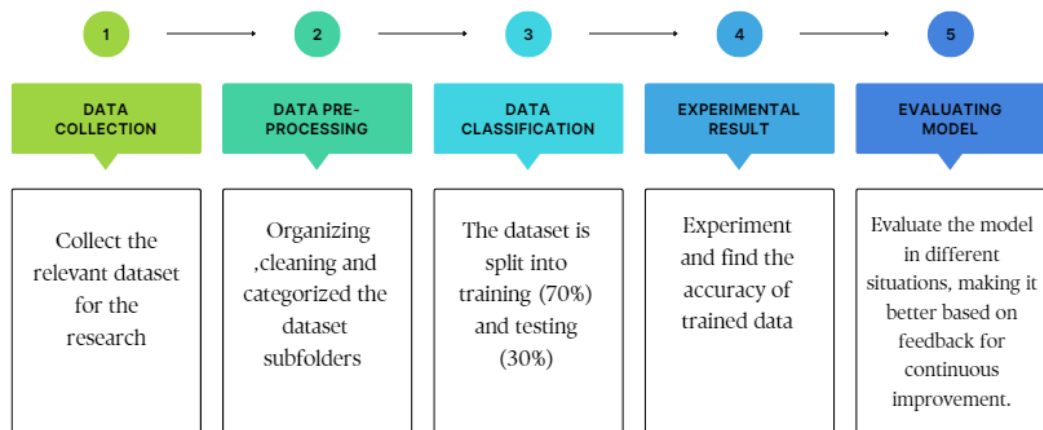


Figure 3.1 Research Framework

3.2.1 Data Collection

The data collection process involves acquiring a diverse image dataset from Kaggle website. The Website offering images capturing a range of driver behaviors, including open eyes, closed eyes, Images were systematically gathered and organized into distinct folders based on the observed behaviors. Special attention was given to ensuring a balanced representation of each category within the dataset.

3.2.2 Data Pre-processing

Following the data collection, the images go through a cleaning and organizing process. Any irregularities or issues in the images are addressed to maintain data quality. The images are standardized for consistency and undergo facial and eye detection to focus on key areas for drowsiness detection. To keep things organized, the images are categorized into subfolders representing open eyes, closed eyes. This step prepares the dataset for training the machine learning model by creating a structured and optimized set of images with clear indicators of drowsiness.

3.2.3 Data Classification

In the data classification stage, the pre-processed images are divided into training and testing sets. The training set, comprising 70% of the images, is utilized to teach the machine learning model to recognize patterns associated with open eyes, closed eyes. The remaining 30% forms the testing set, serving as a separate dataset to evaluate the model's proficiency in identifying drowsiness indicators on new, unseen images. This division into training and testing sets, alongside the specified percentages, ensures a comprehensive assessment of the model's performance and its capacity to generalize to real-world scenarios beyond the training data.

3.2.4 Experimental Result

After training the model, we tested it on new images to see how well it could spot signs of drowsiness. The goal was to figure out how accurate the model is in identifying these signs. We measured its accuracy, which tells us how good it is at getting things right. The higher the accuracy, the better the model is at recognizing drowsiness. This step helps us understand how well the model performs in different situations and where it might need some improvement. The focus is on making sure the model is really good at spotting signs of drowsiness in real-world scenarios.

3.2.5 Evaluating Model

In the final phase, we assess the trained model's real-world performance. Simulated driving scenarios are designed to evaluate its effectiveness under dynamic conditions, accounting for variations in lighting and diverse driver behaviors. User feedback mechanisms gather subjective assessments of the system's usability and practicality in driving situations. The model's responsiveness to real-time conditions and its accuracy in identifying drowsiness indicators are tested interactively. This iterative evaluation process allows for refinements to the model and system design based on real-world feedback, contributing to continuous improvement and optimization. Notably, the Haar Cascade algorithm is employed for real-time face and eye detection, a pivotal component in the system's ability to monitor driver facial and eye activity effectively. This algorithm works by training positive and negative images to detect features like eyes and faces, making it a crucial tool in the system's functionality. The user interaction aspect underscores the practical application of the driver drowsiness detection system, ensuring that research outcomes align with the needs of end-users.

3.3 PROJECT REQUIREMENT

3.3.1 Input

The input for this research involves a carefully curated dataset obtained from Kaggle website from different datasets capturing diverse driver behaviors, including instances of open eyes, closed eyes. This dataset serves as the foundation for training and evaluating the machine learning model developed for driver drowsiness detection. The dataset's richness and variety are critical for ensuring the model's ability to effectively identify and classify different manifestations of drowsiness. The collected images undergo a systematic categorization process, forming distinct folders for open eyes, closed eyes, thereby facilitating a structured approach to training and testing. This dataset not only fuels the development of a robust and accurate model but also reflects the real-world scenarios essential for addressing the pervasive issue of drowsy driving.

3.3.2 Output

The project's outcomes are determined by leveraging the Haar Cascade algorithm in combination with key performance metrics in machine learning, namely accuracy, precision, recall, and F1 score. The Haar Cascade algorithm plays a pivotal role in real-time face and eye detection, a critical component for monitoring driver facial and eye activity. Now, diving into the performance metrics:

$$\begin{aligned} \text{Precision} &= \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} & \text{Recall} &= \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \\ &= \frac{\text{True Positive}}{\text{True Predicted Positive}} & &= \frac{\text{True Positive}}{\text{True Actual Positive}} \end{aligned}$$
$$\text{F1} = 2 \times \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \qquad \text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Figure 3.2: The Evaluation Metrics

Accuracy: This metric measures the overall correctness of the model's predictions. It is the ratio of correctly predicted instances to the total instances.

Precision: Precision assesses the model's accuracy in identifying true positives, minimizing the occurrence of false positives. It is calculated as the ratio of true positives to the sum of true positives and false positives.

Recall: Recall evaluates the model's ability to capture all actual instances of drowsiness, minimizing false negatives. It is calculated as the ratio of true positives to the sum of true positives and false negatives.

F1 Score: The F1 score provides a balance between precision and recall, offering a comprehensive measure of the model's overall performance. It is calculated as the harmonic mean of precision and recall.

These metrics collectively offer a detailed evaluation of the Haar Cascade algorithm's effectiveness in detecting and classifying drowsiness among drivers. They provide valuable insights into the model's accuracy, its ability to avoid false positives and negatives, and the overall reliability in real-world scenarios.

3.3.3 Process Description

The process begins with the acquisition of a diverse image dataset capturing various driver behaviors, including open eyes, closed eyes. The collected images undergo meticulous pre-processing, involving cleaning, organization, and standardization, with a focus on facial and eye detection for drowsiness indicators. Categorized into subfolders, the pre-processed images are then divided into training (70%) and testing (30%) sets for the subsequent classification stage. In this phase, the machine learning model is trained to recognize patterns associated with different behaviors, and its proficiency is evaluated on a separate dataset of new images. Following model training, experimental results are obtained by testing its accuracy in identifying signs of drowsiness, providing insights into its real-world performance. The final evaluation involves subjecting the trained model to simulated driving scenarios, considering dynamic conditions, lighting variations, and diverse driver behaviors. User feedback mechanisms gauge the system's usability and practicality, emphasizing real-world application. Notably, the Haar Cascade algorithm plays a pivotal role in real-time face and eye detection, contributing to the effective monitoring of driver facial and eye activity. The iterative evaluation process allows for continuous refinement and optimization based on real-world feedback, ensuring the alignment of research outcomes with the practical needs of end-users.

3.3.4 Constraints And Limitations

My research has a limitation when it comes to using the Haar Cascade Classifier algorithm for face detection. One issue is that the accuracy of this algorithm depends on a few factors, such as the quality and quantity of the data used to train it, the complexity of the features it's looking for, and how well we adjust its settings. While the algorithm is generally good at finding faces in pictures and videos when trained and set up correctly,

it might not work as well in certain situations. For instance, it could struggle in challenging lighting conditions or when faces are partially covered. It tends to work better in places with good lighting and clear pictures. It's important to be aware of these limitations and evaluate how well the Haar Cascade Classifier algorithm performs in the specific conditions of its use.

3.4 PROPOSED DESIGN

3.4.1 Flowchart

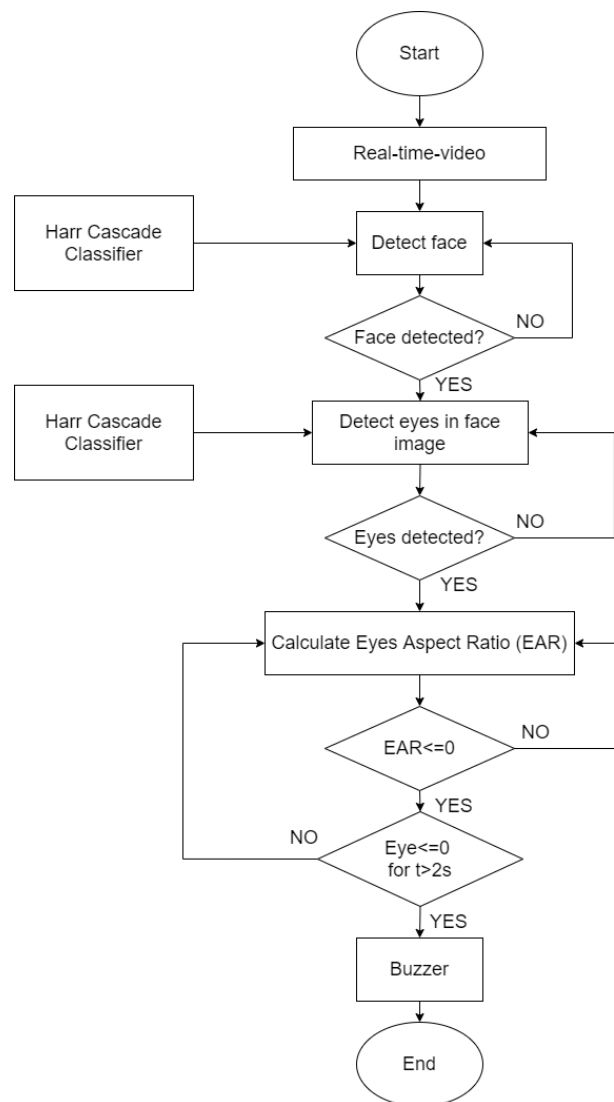


Figure 3.2 Flowchart of proposed design

The process commences with the capture of a real-time video of the driver's face. Subsequently, a Haar Cascade Classifier analyzes the video frame to determine the presence of a face. If a face is detected, the process advances to the next step; otherwise, it returns to the initial stage to capture another video frame. Upon face detection, the Haar Cascade Classifier is employed again to ascertain the presence of eyes in the face image. If eyes are detected, the process proceeds to calculate the Eyes Aspect Ratio (EAR), representing the ratio of the distance between pupils to the distance between the inner and outer corners of the eye. A low EAR signals closed or partially closed eyes, indicative of drowsiness. If the EAR is less than or equal to 0, the process advances; otherwise, it returns to capture another video frame. To identify sustained drowsiness, the process checks if the EAR remains less than or equal to 0 for more than 2 seconds. If so, it triggers an alarm, activating a buzzer to alert the driver. The process concludes once the buzzer is activated.

3.5 GANTT CHART

The Gantt Chart shows the progress of the research with the timescale. The research will be following the proposed methodology. The Gantt Chart can be referred to APPENDIX A.

CHAPTER 4

IMPLEMENTATION, RESULTS AND DISCUSSION

4.1 INTRODUCTION

The purpose of this chapter is to offer a comprehensive explanation of the implementation and results obtained in the research paper on driver sleep detection using machine learning with the Haar Cascade algorithm.

4.2 EVIDENCE OF EARLY WORK

In the early stages of the project, the initial code serves as foundational evidence for the development of the driver sleep detection system using the Haar Cascade algorithm. Referring to Appendix B provides a detailed insight into the early coding efforts that set the stage for the successful implementation of the research.

4.3 DATA DESIGN

4.3.1 Data Description

I obtained the dataset from the Kaggle website, which exclusively comprises images. The dataset is composed of two distinct files, one containing images of open eyes and the other featuring closed eyes. Each file contains a total of 2000 images, providing a comprehensive representation of both eye states. Prior to utilization, the dataset has undergone pre-processing to enhance its quality and usability for analytical purposes. This preparatory step ensures that the dataset is optimized for subsequent tasks, contributing to the reliability and accuracy of any analyses or models developed using this curated collection of eye images (PRASAD V PATIL, 2020).

Drowsiness Detection Dataset

Classify based on whether Eyes are Closed or Open.



Data Card Code (15) Discussion (0)

About Dataset

The **Drowsiness Detection Dataset** is generated using MRL and Closed Eyes in Wild(CEW) dataset, as well as our own unique dataset. This large-scale dataset comprising of both closed and open human eye images can be majorly used for eye detection and further be extended for drowsiness detection.

Images from the dataset were taken under a variety of conditions, including diverse lighting conditions, distance, resolution, face angle, and eye angle. These parameters aid in obtaining good outcomes with minimal chances of getting low accuracy.

There are various versions of this dataset, with **Version 1** having **10k** images split into 5,000 images for closed eyes and 5,000 images for open eyes. The **5k** images in **Version 2** are split into 2,500 images for closed eyes and 2,500 images for open eyes. Another collection of **10k** images, divided into 5,000 images for each of the closed and open eyes, may be found in **Version 3**. 2,000 images of closed eyes and 2,000 images of open eyes make up the 4th version of the dataset i.e. **Version 4** with **4k** images in total.

Usability 8.75

License
CC0: Public Domain

Expected update frequency
Never

Tags

Computer Vision
Deep Learning CNN

Figure 4.1 : Dataset description

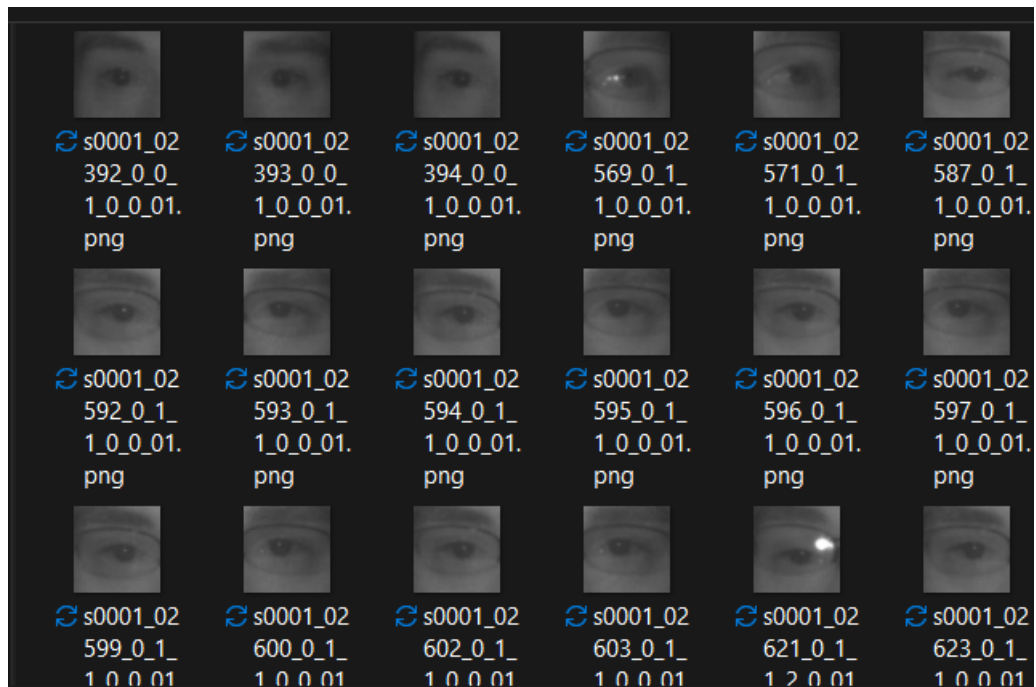


Figure 4.2: Dataset conten

4.4 PROOF OF INITIAL CONCEPT

4.4.1 Haar Cascade Classifier

The Haar Cascade Classifier stands out among object detection methods for its capability to identify faces swiftly. Its efficiency lies in high-speed computation, determined by the number of pixels within a rectangle feature rather than individual pixel values. This method involves four key steps for object detection: Haar-like feature analysis, integral image computation, AdaBoost learning, and Cascade Classifier application (Doifode, V, 2023). In face detection, Haar features play a pivotal role. They are utilized to identify the presence of features in a given image, generating a single value for each feature by summing pixels under a black rectangle. These Haar-like features, characterized by rectangular patterns, offer specific cues for rapid face detection within an image (Kamarudin et al., 2019).

4.4.1.1 Haar-like features

Haar-like features are utilized to detect features present in an image. This method employs square functions that provide distinctive indications within an image. These features are comprised of various rectangular configurations, divided into four sections, as illustrated in Figure 4.3.



Figure 4.3: Haar-like Features

Haar-like features, as illustrated in Figure 3.5, are made out of a rectangular arrangement of two rectangles, three rectangles, and four rectangles separated into four features. The difference in the number of pixels in the area of the two rectangles is used

to determine the value of the features of two rectangles in Haar-like features. The rectangle's area has the same form when placed next to each other vertically or horizontally. The computation for the three-rectangle feature is done by adding the two outside squares, then subtracting the result from the total of the rectangles in the middle. The value in rectangular features is derived by calculating the difference between the diagonal pairings (Kamarudin et al., 2019).

4.4.1.2 Integral Image

Integral images are utilised to expedite the computational outcomes of extracting values from haar-like features after employing haar-like features to search for features in the picture. As seen in Figure 4.4 a, it begins scanning the picture from the upper left corner to find faces, and it concludes the face detection process at the right bottom of the image.

As shown in Figure 4.4 b, the integral image could compute values rather quickly and correctly by generating a new picture presentation based on the value of an area that had previously been scanned by a particular Haar-like feature (Eryl Nanda Pratama & Maki, 2022).

$$I(x, y) = i(x, y) + I(x, y - 1) + I(x - 1, y) + I(x - 1, y - 1)$$

	1	2	3	4
1	3	7	7	3
2	1	3	3	1
3	5	9	9	5
4	3	6	6	3

(a)

	1	2	3	4
1	3	10	17	20
2	4	14	24	28
3	9	28	47	56
4	12	37	62	74

(b)

Figure 4.4: (a) Input image - (b) Integral image

$$(1) \quad I(x, y) = \sum_{\substack{x' \leq x \\ y' \leq y}} i(x', y')$$

$$(2) \quad I(x, y) = i(x, y) + I(x, y - 1) + I(x - 1, y) + I(x - 1, y - 1)$$

$$(3) \quad \sum_{\substack{x_0 < x < x_1 \\ y_0 < y < y_1}} i(x, y) = I(D) + I(A) - I(B) - I(C)$$

Figure 4.5: Equations

Equation (1) illustrates that the value at any given place (x,y) is the summed area table of the total of all the pixels above and to the left of (x,y), inclusive. where I(x,y) is the total of the integral of the pixel values and i(x,y) is the value of the pixel at (x,y). The

integral image value, $I(x,y)$, is found by adding the preceding index values from top left to bottom right. Additionally, the value in the summed-area table at (x,y) in Equation (2) may be calculated effectively in a single pass over the picture (Eryl Nanda Pratama & Maki, 2022). Regardless of the area size, calculating the sum of intensities across any rectangular area takes precisely four array references after the summed-area table has been calculated. The total of $i(x,y)$ across the rectangle that is covered by A, B, C, and D is displayed in equation (3).

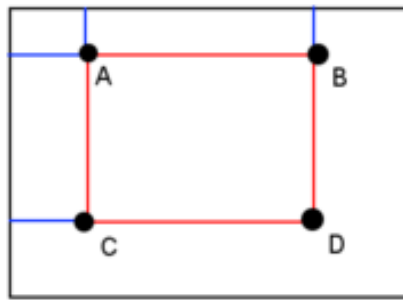


Figure 4.5: Description of computing a sum in the summed-area table

The notation in Figure 4.6 shows the example of the description of computing a sum in the summed-area table data structure having $A=(x_0,y_0)$, $B=(x_1,y_0)$, $C=(x_0,y_1)$ and $D=(x_1,y_1)$.

4.4.1.3 AdaBoost

Adaboost uses tiny feature sets and classifier training to improve classification performance. Improving the classification performance of basic or subpar training procedures was the original goal of the AdaBoost algorithm (Doifode, V, 2023). To boost AdaBoost's performance, Viola Jones recommends applying the brute force method, which comprises identifying the weakest classifier by carefully examining each feature in the training set to find the one that works the best. Weak classifiers connected linearly by AdaBoost produce a strong classifier (Gao. Z,2022).

4.4.1.4 Cascade Classifier

Although it takes less time to compute, the cascade classifier improves detection performance (Kamarudin et al., 2019). This algorithm's categorization process involves many stages, each of which generates sub-images that are deemed unrelated to eyes. Figure 4.7 shows the Cascade classifier's structure as follows:

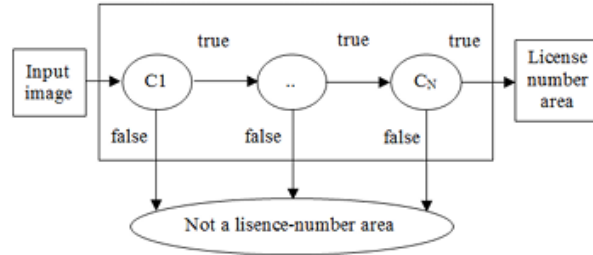


Figure 4.6: Cascade Classifier Structure

The input picture will proceed to the first level of categorization, as seen in figure 4.7. A cascade classifier is used to enter the stratified classification process for each sub-image (image input at each level) using the findings of the Adaboost strong classifier. The categorization for each level will indicate whether or not the given sub-image contains a face. If so, the sub picture will not be able to advance to the next level and will be discarded (Zhao, Y., Xie,2020).

4.4.2 Eye Aspect Ratio (EAR)

The driver's degree of drowsiness is determined by first identifying their face and then using their eye blink rate. The scalar value may be used by the Eye Aspect Ratio (EAR) formula, which was developed in (Doifode, V, 2023), to identify eye blinks. For example, if a driver blinks their eyes more often, it indicates that they are sleepy. Therefore, in order to determine the frequency of eye blinking, a precise detection of the eye's shape is required. The EAR is used to evaluate the state of eye openness based on landmarks found in the face-containing picture. The eye landmarks between the calculated eye's height and width are identified for each video frame. Equation (4) may be used to define the eye aspect ratio.

$$(4) \quad EAR = \left\lceil \frac{||P2-P6|| + ||P3-P5||}{2||P1-P4||} \right\rceil \quad (5) \quad EAR = \begin{pmatrix} X > 0; & \text{eyes open} \\ 0; & \text{eyes closed} \end{pmatrix}$$

Figure 4.7: EAR Equations

The eye aspect ratio formula is shown in equation (4), where the 2D landmark positions are p1 through p6. As seen in Figure 3.11 a, the p2, p3, p5, and p6 are used to measure height, while the p1 and p4 are used to measure eye width in meters (m). As seen in Figure 3.11 b, the eye aspect ratio remains constant when the eye is open but rapidly decreases to about 0 when the eye is closed. The EAR output range while the eyes are open and closed is displayed in equation (5). When the eyes are closed, the EAR result is about zero, but when the eyes are open, it can be any integer value that is more than zero.

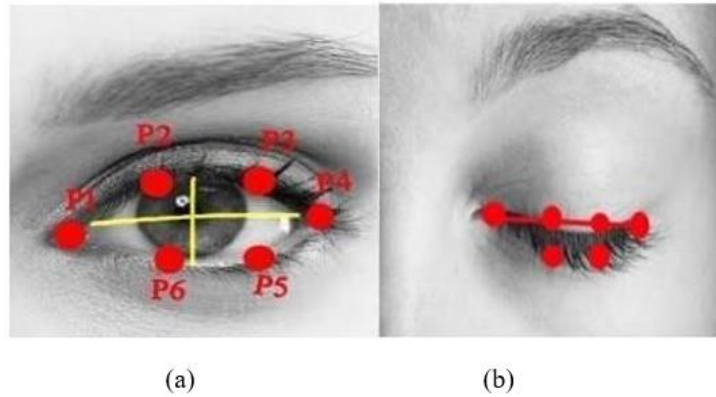


Figure 4.8: Landmarks of Open and Close eye

4.4.3 Support Vector Machine (SVM)

An SVM is a powerful machine learning tool that tackles classification, regression, and outlier detection tasks by finding the best way to separate data points into distinct groups or categories. It's used extensively in various fields like natural language processing, and image recognition to solve complex problems by determining the most effective boundaries between different data points (Nanda Pratama, 2022).

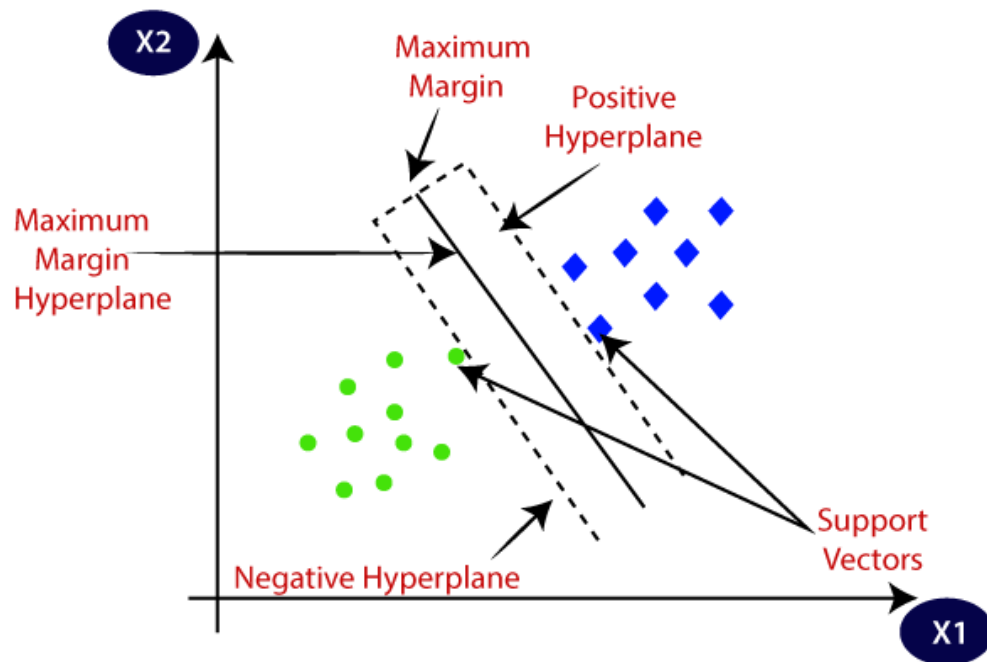


Figure 4.9 SVM

SVM stands out as an effective classification algorithm in supervised learning. Its primary function involves categorizing data into distinct classes based on labeled training data. A key strength of SVM lies in its versatility, capable of addressing both classification and regression tasks. It accomplishes this by creating a hyperplane decision boundary between classes, enabling effective separation or classification. Beyond classification, SVM finds utility in applications such as object detection and image classification.

4.4.3.1 How does SVM work?

The main objective is to segregate the given dataset in the best possible way. The distance between the nearest point is known as the margin. The objective is to select a hyperplane with the maximum possible margin between support vectors in the given dataset. SVM searches for the maximum marginal hyperplane in the following steps:

Generate hyperplanes that segregate the classes in the best way. Left-hand side figure showing three hyperplanes black, blue, and orange. Here, the blue and orange have higher classification errors, but the black is separating the two classes correctly.

1. Select the right hyperplane with the maximum segregation from the nearest data points as shown in the right-hand side figure.

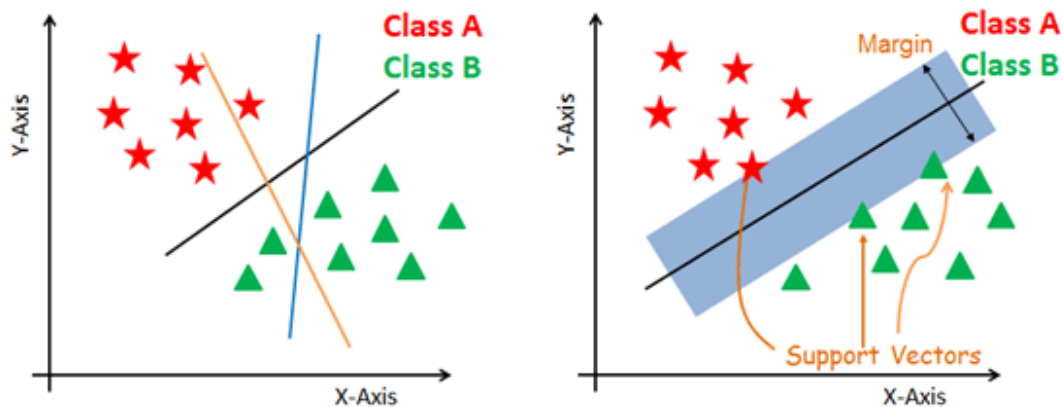


Figure 4.10 SVM hyperplane

4.4.4 K-Nearest Neighbor (KNN) Algorithm

KNN is one of the most fundamental but critical classification algorithms in machine learning. It is heavily used in pattern recognition, data mining, and intrusion detection and is a member of the supervised learning domain. Since it is non-parametric, which means it does not make any underlying assumptions about the distribution of data (unlike other algorithms like GMM, which assume a Gaussian distribution of the provided data), it is extensively applicable in real-life circumstances. An attribute-based previous data set (also known as training data) is provided to us, allowing us to classify locations into groups.

Take into consideration, for instance, the table of data points with two properties below:

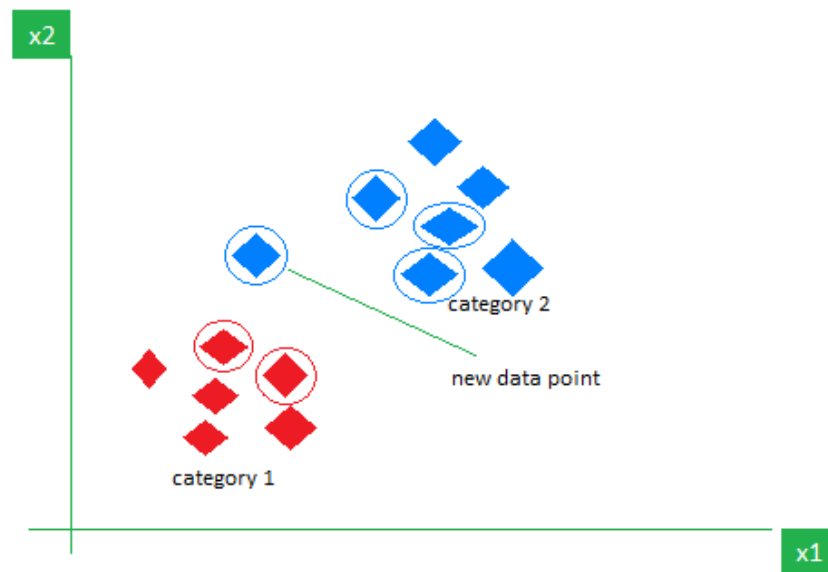


Figure 4.11 KNN Algorithm working visualization

Now, given another set of data points (also called testing data), allocate these points to a group by analyzing the training set. Note that the unclassified points are marked as 'White'.

4.4.4.1 How Does the KNN Algorithm Work

The K Nearest Neighbor (KNN) algorithm is a simple yet powerful method used for both classification and regression tasks. Here's how it works:

1. **Training Phase:** During the training phase, KNN stores all available data points and their corresponding class labels (for classification) or target values (for regression). KNN doesn't explicitly learn a model; instead, it memorized the entire training dataset (Sudarshan, P, 2023).
2. **Prediction Phase:** When a new, unseen data point is presented for prediction, KNN identifies the K nearest neighbors to that data point from the training dataset. Nearest neighbors are determined based on a distance metric, typically Euclidean distance, though other metrics like Manhattan

distance can be used. The value of K is predetermined and represents the number of neighbors to consider. It is usually an odd number to avoid ties.

3. **Classification:** For classification tasks, KNN predicts the class label of the new data point by selecting the majority class among its K nearest neighbors. This is done through a majority voting scheme, where each neighbor gets one vote, and the class with the most votes is assigned to the new data point.
4. **Regression:** For regression tasks, KNN predicts the target value of the new data point by averaging the target values of its K nearest neighbors. Alternatively, weighted averaging can be used, where closer neighbors have a higher influence on the prediction.
5. **Decision Boundary:** KNN does not explicitly learn a decision boundary; instead, it forms decision regions based on the distribution of the training data. Decision boundaries are non-linear and depend on the arrangement of data points in the feature space.
6. **Distance Metric:** The choice of distance metric is crucial and depends on the nature of the data. Euclidean distance is commonly used for continuous features, while other metrics like Manhattan distance may be more suitable for categorical features.
7. **Scaling:** Feature scaling is often necessary to ensure that all features contribute equally to the distance calculations. Common scaling techniques include normalization or standardization.
8. **Computational Complexity:** The computational complexity of KNN grows linearly with the size of the training dataset since it requires calculating distances between the new data point and all training instances.
9. **Choosing K :** Selecting an appropriate value for K is important. A small K can lead to overfitting, while a large K may result in underfitting. Cross-validation techniques are often used to determine the optimal value of K .

Overall, KNN is a versatile and intuitive algorithm, particularly suitable for datasets with low dimensionality and non-linear relationships between features.

4.4.4.2 Advantages of the KNN Algorithm

- Easy to implement as the complexity of the algorithm is not that high.
- **Adapts Easily** – As per the working of the KNN algorithm it stores all the data in memory storage and hence whenever a new example or data point is added then the algorithm adjusts itself as per that new example and has its contribution to the future predictions as well.
- **Few Hyperparameters** – The only parameters which are required in the training of a KNN algorithm are the value of k and the choice of the distance metric which we would like to choose from our evaluation metric.

4.4.4.3 Disadvantages of the KNN Algorithm

- **Does not scale** – As we have heard about this that the KNN algorithm is also considered a Lazy Algorithm. The main significance of this term is that this takes lots of computing power as well as data storage. This makes this algorithm both time-consuming and resource exhausting.
- **Curse of Dimensionality** – There is a term known as the peaking phenomenon according to this the KNN algorithm is affected by the curse of dimensionality which implies the algorithm faces a hard time classifying the data points properly when the dimensionality is too high.
- **Prone to Overfitting** – As the algorithm is affected due to the curse of dimensionality it is prone to the

4.5 HARDWARE AND SOFTWARE EQUIPMENT

Below show hardware and software requirements that will be utilized in this research paper.

Table 2: Hardware for the research

Hardware	Specification	Purpose
Laptop	Acer Nitro 5 CPU: 9h Gen. Intel® Core™ i5 Processor OS: Windows 11	For developing, documenting, and completing research
Smartphone	Redmi note 11 pro Processor: Octa-core (2x2.05 GHz Cortex-A76) Ram: 6GB OS: Android 11, MIUI 13	To aid in finding materials and information needed to complete the project

Table 3: Software for research

Software	Specification	Purpose
Microsoft Office Word	Version 2020	Used for research documentation
Draw io	Version 18.0.3	Used to draw flowchart or framework

Google Chrome	Version 102.0.5005.62	Assists in finding previous research papers and materials related to the project
Spyder	Python 3.9.13	Run Python code for research purposes

4.6 PSEUDOCODE

In the advanced stages of the project, the final code marks a significant milestone in the development of the driver sleep detection system. The trained model, detailed in Appendix C, and the real-time detection implementation, outlined in Appendix D, demonstrate the culmination of extensive research and development efforts. These sections provide comprehensive insights into the final coding processes that led to the successful realization of the project.

```
import cv2
import time
import os
import numpy as np
from tensorflow.keras.models import load_model
```

Figure 4.12: Imports and Model Loading

The necessary libraries are imported:

- cv2 for computer vision tasks.
- time for handling time-related functions.
- os for interacting with the operating system.
- numpy for numerical operations.
- load_model from tensorflow.keras to load the pre-trained drowsiness detection model.

```
model = load_model('drowsiness_detection_model.h5')
```

Figure 4.13: Load the Pre-trained Model

The pre-trained model named *drowsiness_detection_model.h5* is loaded using the *load_model* function. This model is used to predict whether the driver's eyes are open or closed.

```
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')  
eye_cascade = cv2.CascadeClassifier('haarcascade_eye_tree_eyeglasses.xml')
```

Figure 4.14: Load the Haar Cascade Classifiers

Haar Cascade classifiers for detecting faces and eyes are loaded using *cv2.CascadeClassifier*. These XML files contain trained models for face and eye detection.

```
cap = cv2.VideoCapture(0)  
a = 0  
b = 0  
c = time.time()  
no_eyes_counter = 0  
no_eyes_threshold = 2 # Number of consecutive "no eyes" detections before alert
```

Figure 4.15: Initialize Video Capture and Variables

- Video capture from the webcam is initialized with *cv2.VideoCapture(0)*.
- Several variables are initialized:

a and b for counting closed and open eye detections, respectively.

`c` to store the current time for interval checks. `no_eyes_counter` to count consecutive frames with no eye detections. `no_eyes_threshold` to set the threshold for triggering an alert if no eyes are detected consecutively.

```
def preprocess_eye(eye_img):  
    eye_img = cv2.resize(eye_img, (224, 224))  
    eye_img = eye_img.astype('float32') / 255.0  
    eye_img = np.expand_dims(eye_img, axis=0)  
    return eye_img
```

Figure 4.16: Preprocess Eye Function

This function preprocesses the eye images before feeding them to the model:

- The image is resized to 224x224 pixels.
- Pixel values are normalized by dividing by 255.0.
- The image is expanded along the first dimension to match the model's input shape, creating a batch size of one.

```
while True:  
    ret, img = cap.read()  
    if not ret:  
        break
```

Figure 4.17: Main Loop for Video Capture

- The main loop captures frames from the webcam continuously.

`ret` indicates if the frame was captured successfully.

If `ret` is False, the loop breaks, ending the video capture.

```

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.3, 5)

if len(faces) == 0:
    print("No face detected")

```

Figure 4.18: Convert Frame to Grayscale and Detect Faces

- The captured frame is converted to grayscale using `cv2.cvtColor`.
- Faces are detected in the grayscale image using the `face_cascade`.
- If no faces are detected, a message is printed.

```

for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)

    roi_gray = gray[y:y + h, x:x + w]
    roi_color = img[y:y + h, x:x + w]

    eyes = eye_cascade.detectMultiScale(roi_gray)

```

Figure 4.19: Process Each Detected Face

- For each detected face, a rectangle is drawn around the face in the original frame.
- Regions of interest (ROI) for the grayscale and color images are defined based on the face coordinates.
- Eyes are detected within the face ROI using the `eye_cascade`.

```

if (time.time() - c) >= 15:
    if a / (a + b) >= 0.2:
        os.system('buzz.mp3')
        print("****ALERT****", a, b, a / (a + b))
    else:
        print("safe", a / (a + b))
    c = time.time()
    a = 0
    b = 0

```

Figure 4.20: Check Drowsiness time

- Every 2 seconds, the ratio of closed-eye detections (a) to total detections (a + b) is checked.
- If the ratio is greater than or equal to 0.2, an alert sound is played using `os.system('buzz.mp3')`.
- The counters and timestamp are reset.

```

if len(eyes) == 0:
    a += 1
    no_eyes_counter += 1
    print('no eyes!!!')
    if no_eyes_counter >= no_eyes_threshold:
        os.system('buzz.mp3')
        print("****ALERT: No eyes detected for three consecutive frames****")
        no_eyes_counter = 0 # Reset the counter after alert
else:
    no_eyes_counter = 0 # Reset the counter if eyes are detected
    for (ex, ey, ew, eh) in eyes:
        eye_img = roi_color[ey:ey+eh, ex:ex+ew]
        eye_img_preprocessed = preprocess_eye(eye_img)
        prediction = model.predict(eye_img_preprocessed)
        if prediction < 0.5:
            a += 1
            print('Closed eyes detected')
        else:
            b += 1
            print('Open eyes detected')
    cv2.rectangle(roi_color, (ex, ey), (ex + ew, ey + eh), (0, 255, 0), 2)

```

Figure 4.21: Detect Closed Eyes and Handle No Eyes Detected

- If no eyes are detected, the closed-eye counter `a` and `no_eyes_counter` are incremented.
- If `no_eyes_counter` exceeds the threshold, an alert sound is played, and the counter is reset.
- If eyes are detected, the `no_eyes_counter` is reset. Each detected eye is processed:
- The eye image is preprocessed using `preprocess_eye`.
- The model predicts whether the eye is open or closed.
- The counters `a` (closed eyes) or `b` (open eyes) are incremented based on the prediction.
- A rectangle is drawn around each detected eye.

```
cv2.imshow('img', img)

k = cv2.waitKey(1) & 0xff
if k == 27:
    print("noeyes: ", a)
    print("total: ", (a + b))
    break
```

Figure 4.22: Display the Processed Frame

- The processed frame is displayed using `cv2.imshow`.
- The loop continues until the "Esc" key (ASCII value 27) is pressed.
- Upon exiting, the total counts of closed-eye detections are printed.

```
cap.release()  
cv2.destroyAllWindows()
```

Figure 4.23: Release Resources

- The video capture is released using *cap.release()*.
- All OpenCV windows are closed using *cv2.destroyAllWindows()*.

4.7 TESTING AND EVALUATION

The test and evaluation part is where we put everything to the test. We want to see if our experiment works well when we bring all its parts together. This is important to solve the problem we identified and find ways to improve based on what we learn. The main goal of this test is to show that our real-time processing sleep detection system is the best for what we want to achieve. We also look closely at any issues or mistakes we find to make our project even better in the future.

After that, we carefully study the results of our experiment with the detection system. We use these findings to write a simple conclusion, checking if our initial idea was right. This concludes the detailed documentation of our project, showing that we've achieved what we set out to do. The next chapter will include a further description of the implementation process in more depth.

4.8 POTENTIAL USED OF PROPOSED SOLUTION

The proposed solution in my research, a real-time driver sleep detection system, offers versatile applications. First and foremost, it significantly enhances road safety by actively identifying signs of driver fatigue, preventing potential accidents. This solution holds promise for integration into the automotive industry, contributing to advancements in autonomous driving and serving as an added safety feature. Additionally, its applications extend to transportation and logistics, fleet management, public transportation safety, and even potential integration into the insurance industry, influencing safety-based premiums. The system's adaptability also allows for

personalized use in individual vehicles, providing customizable alerts for drivers with varying needs.

Moreover, beyond its immediate safety applications, the system's data collection capabilities hold potential for research on drowsy driving patterns. The insights gained could contribute to a deeper understanding of driver fatigue, supporting broader efforts to address this critical safety issue in various contexts. Overall, the proposed solution has the potential to significantly impact safety across multiple industries and contribute to ongoing initiatives for safer and more efficient transportation systems.

4.9 RESULT

In this research on driver sleep detection using machine learning, a dataset was procured from Kaggle, comprising images categorizing instances of open eyes and closed eyes. The dataset underwent meticulous cleaning and organizing, followed by a division where 70% was allocated for training and 30% for testing. Various machine learning algorithms, including Haar Cascade, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM), were evaluated in this study.

For the evaluation phase, key metrics were employed to assess the performance of each machine learning model. These metrics include accuracy, precision, recall, and F1 score. Accuracy gauges the overall correctness of the model, precision measures its ability to avoid false positives, recall assesses its capability to find all relevant instances, and the F1 score provides a balanced evaluation of precision and recall. By comparing these metrics across the different algorithms, the research aims to identify the most effective solution for detecting driver drowsiness. The algorithm yielding the best results based on these comprehensive metrics will be chosen for implementation.

Table 4: Accuracy, Precision, Recall and F1_Score of Machine Learning Algorithms

Model	Open Eyes			Closed Eyes			Accuracy
	Precision	Recall	F1_score	Precision	Recall	F1_score	
Harr Cascade	99%	71%	83%	79%	99%	88%	86%
KNN	65%	69%	67%	70%	67%	68%	68%
SVM	84%	72%	78%	75%	86%	80%	79%

The table 4 presents the performance metrics of three different machine learning algorithms—Haar Cascade, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM)—used for detecting driver drowsiness by distinguishing between open eyes and closed eyes. The metrics evaluated include precision, recall, and F1 score for both open eyes and closed eyes, as well as the overall accuracy of each algorithm.

Precision indicates the accuracy of positive predictions, with higher precision meaning fewer false positives. Recall measures the algorithm's ability to find all positive instances, where higher recall indicates fewer false negatives. The F1 score is the harmonic mean of precision and recall, providing a balanced measure of both. Accuracy reflects the overall correctness of the model across all predictions.

The Haar Cascade algorithm demonstrates outstanding performance with an overall accuracy of 86%. For open eyes, it achieves a high precision of 99%, indicating very few false positives, though it has a lower recall of 71%, showing some false negatives. The F1 score for open eyes is 83%, balancing precision and recall. For closed eyes, Haar Cascade shows a precision of 79% and an exceptional recall of 99%, indicating excellent detection with very few missed instances, resulting in an F1 score of 88%.

In contrast, the KNN algorithm exhibits moderate performance with an overall accuracy of 68%. For open eyes, KNN has a precision of 65% and a recall of 69%, resulting in an F1 score of 67%. For closed eyes, the precision is 70% and recall is 67%, leading to an F1 score of 68%.

The SVM algorithm performs better than KNN but not as well as Haar Cascade, with an overall accuracy of 79%. For open eyes, SVM achieves a precision of 84% and a recall of 72%, yielding an F1 score of 78%. For closed eyes, the precision is 75% and the recall is 86%, resulting in an F1 score of 80%.

In conclusion, the Haar Cascade algorithm outperforms both KNN and SVM in terms of accuracy and F1 scores, making it the most effective algorithm for this application. It demonstrates high precision and recall for both open and closed eyes, contributing to its high overall accuracy and robust performance in detecting driver drowsiness.

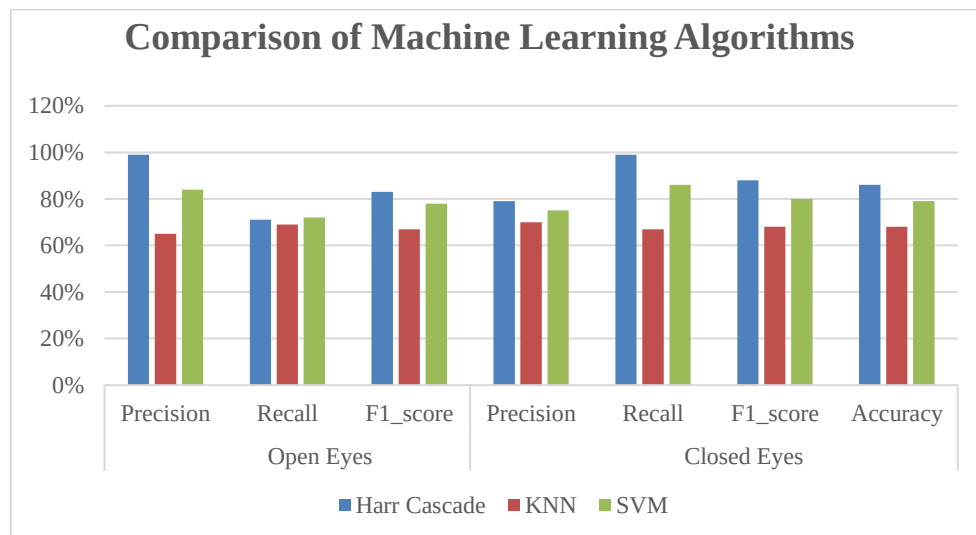


Figure 4.24: Comparison of Machine Learning Algorithms

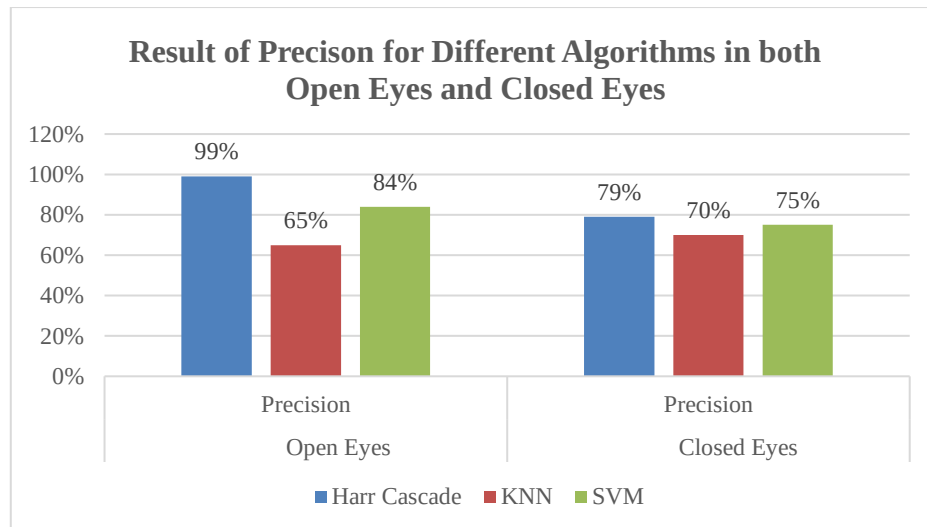


Figure 4.25: Result of Precision for Different Algorithms in both Open Eyes and Closed Eyes

Figure above shows the precision for detecting driver drowsiness based on open and closed eyes, Haar Cascade algorithm excels with 99% precision for open eyes and 79% for closed eyes, indicating minimal false positives and reliable detection. K-Nearest Neighbors (KNN) follows with 65% precision for open eyes and 70% for closed eyes, showing moderate accuracy but higher false positives compared to Haar Cascade. Support Vector Machine (SVM) achieves 84% precision for open eyes and 75% for closed eyes, displaying good accuracy yet slightly lower than Haar Cascade overall. Haar Cascade stands out for its precision in distinguishing eye states, making it highly effective for this application compared to KNN and SVM.

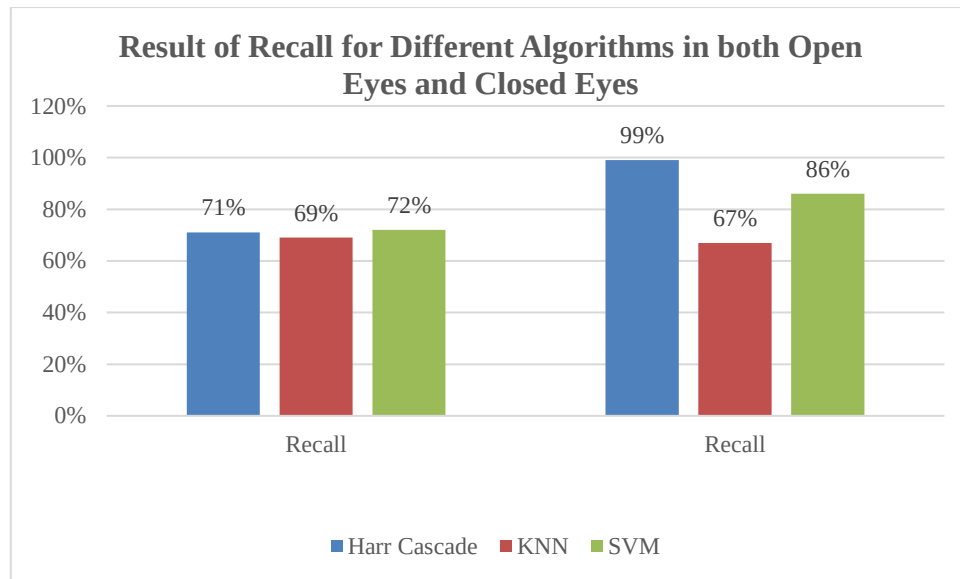


Figure 4.26: Result of Recall for Different Algorithms in both Open Eyes and Closed Eyes

Figure above shows the recall for detecting driver drowsiness based on open and closed eyes, Haar Cascade algorithm achieves a high recall of 71% for open eyes and an exceptional 99% for closed eyes, indicating strong detection capabilities with minimal missed instances of closed eyes. K-Nearest Neighbors (KNN) follows with 69% recall for open eyes and 67% for closed eyes, showing moderate performance in identifying eye states. Support Vector Machine (SVM) demonstrates 72% recall for open eyes and a robust 86% for closed eyes, indicating effective detection across both states. Overall, Haar Cascade excels in detecting closed eyes, while SVM performs strongly in recall for both open and closed eyes compared to KNN and Haar Cascade.

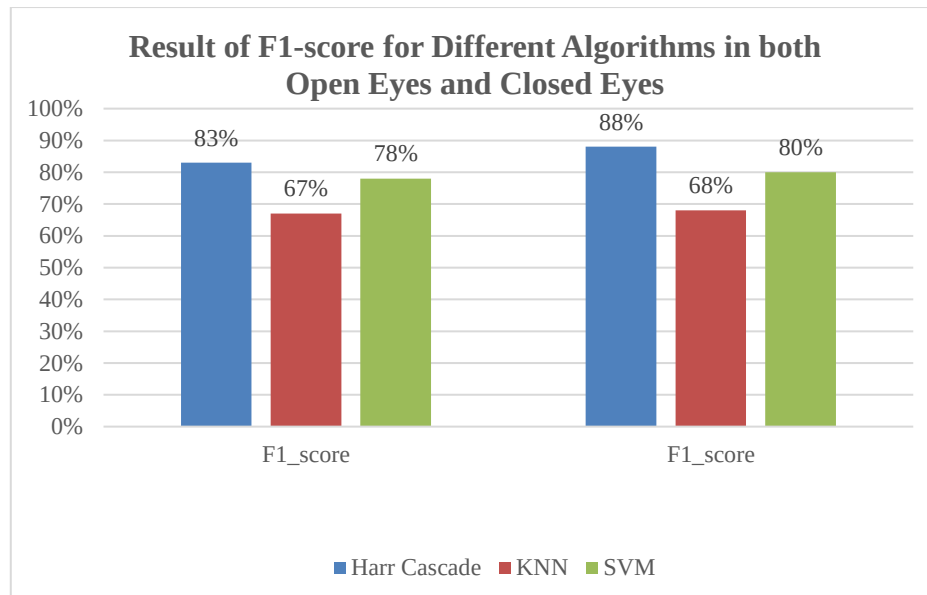


Figure 4.27: Result of F1-score for Different Algorithms in both Open Eyes and Closed Eyes

Figure above shows the F1-score for detecting driver drowsiness based on open and closed eyes, Haar Cascade algorithm achieves strong performance with F1-scores of 83% for open eyes and 88% for closed eyes, indicating a robust balance between precision and recall. K-Nearest Neighbors (KNN) follows with F1-scores of 67% for open eyes and 68% for closed eyes, showing moderate effectiveness in achieving a balance between precision and recall. Support Vector Machine (SVM) demonstrates good F1-scores of 78% for open eyes and 80% for closed eyes, indicating reliable performance in accurately identifying both eye states. Haar Cascade excels particularly in F1-score for closed eyes, while SVM performs consistently well across both eye states compared to KNN and Haar Cascade.

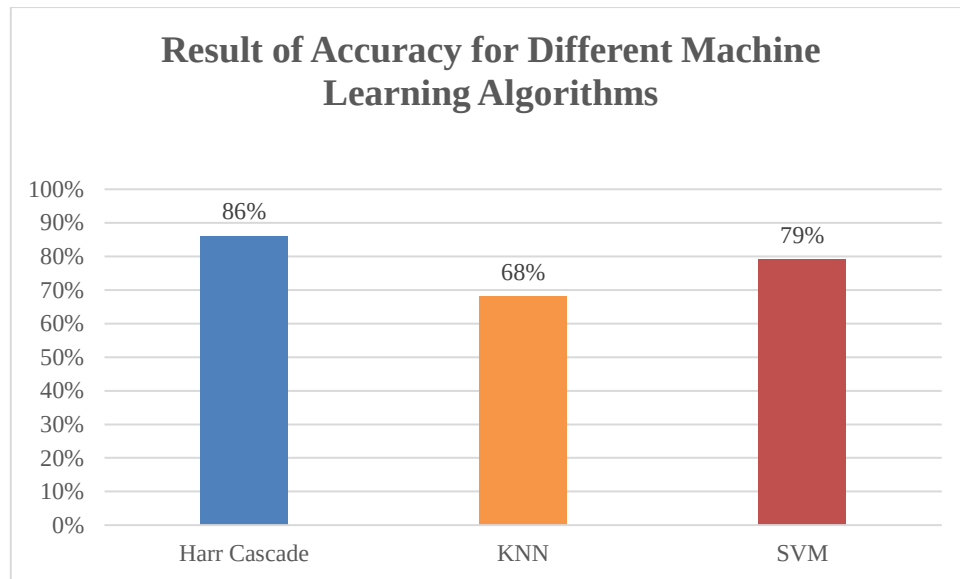


Figure 4.28: Result of Accuracy for Different Machine Learning Algorithms

Figure above shows the results of the accuracy for detecting driver drowsiness using different algorithms. Haar Cascade achieves the highest accuracy at 86%, indicating strong performance with minimal errors in detecting drowsiness. K-Nearest Neighbors (KNN) follows with 68% accuracy, showing moderate effectiveness compared to both Haar Cascade and SVM. Support Vector Machine (SVM) demonstrates an accuracy of 79%, performing well in accurately classifying drowsiness. Haar Cascade's robust accuracy makes it a reliable choice for this application compared to KNN and competitive with SVM.

Truth	Close Eyes	595	5
	Open Eyes	172	428
		Close Eyes	Open Eyes
		Prediction	

Figure 4.29: Confusion Matrix of Harr Cascade

The confusion matrix for Haar Cascade algorithm showcases its performance in detecting driver drowsiness based on open and closed eyes. Out of 600 actual instances of closed eyes, Haar Cascade correctly predicted 595 instances (99%) and incorrectly predicted 5 instances (1%) as open eyes. For 600 actual instances of open eyes, Haar Cascade correctly predicted 428 instances (71%) and incorrectly predicted 172 instances (29%) as closed eyes. This matrix underscores Haar Cascade's exceptional accuracy in identifying closed eyes, with a high rate of correct predictions and minimal false positives. However, it shows a comparatively higher rate of false negatives for open eyes, suggesting room for improvement in detecting this state accurately.

Truth	Close Eyes	517	83
	Open Eyes	164	436
		Close Eyes	Open Eyes
		Prediction	

Figure 4.30: Confusion Matrix of KNN

The confusion matrix for K-Nearest Neighbors (KNN) illustrates its performance in detecting driver drowsiness based on open and closed eyes. It shows that out of 600 actual instances of closed eyes, KNN correctly predicted 517 instances (86%) and incorrectly predicted 83 instances (14%) as open eyes. For 600 actual instances of open eyes, KNN correctly predicted 436 instances (73%) and incorrectly predicted 164 instances (27%) as closed eyes. This matrix provides a clear breakdown of KNN's ability to classify eye states, highlighting its strengths and areas for improvement in identifying drowsiness indicators.

Truth	Close Eyes	424	176
	Open Eyes	208	392
		Close Eyes	Open Eyes
		Prediction	

Figure 4.31: Confusion Matrix of SVM

The confusion matrix for Support Vector Machine (SVM) depicts its performance in detecting driver drowsiness based on open and closed eyes. Out of 600 actual instances of closed eyes, SVM correctly predicted 424 instances (71%) and incorrectly predicted 176 instances (29%) as open eyes. For 600 actual instances of open eyes, SVM correctly predicted 392 instances (65%) and incorrectly predicted 208 instances (35%) as closed eyes. This matrix provides a detailed overview of SVM's effectiveness in classifying eye states, highlighting its strengths in correctly identifying closed eyes but also noting its higher misclassification rate for open eyes.

4.10 DISCUSSION

In conclusion, based on the results from the algorithms—Haar Cascade, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM)—for detecting driver drowsiness, Haar Cascade demonstrates superior accuracy in identifying closed eyes with minimal errors, although it shows a higher rate of false negatives for open eyes compared to SVM. KNN exhibits moderate performance overall, while SVM shows balanced accuracy and recall for both eye states. The successful application of the Haar Cascade algorithm underscores its reliability in detecting driver fatigue, offering a promising solution to mitigate the risks associated with drowsy driving. These findings contribute valuable insights to the field of driver monitoring systems, emphasizing the significance of machine learning techniques for proactive road safety measures.

CHAPTER 5

CONCLUSION

5.1 OBJECTIVE REVISITED

For this thesis, the objective has been to develop a driver sleep detection and alarming system using machine learning techniques. Throughout the research process, Haar Cascade classifiers within the OpenCV library were identified as the most suitable technique for real-time monitoring of a driver's facial features. The system's accuracy in detecting drowsiness was enhanced by fine-tuning the classifier and training it on a diverse dataset. This approach ensured robust performance under various conditions. By using the developed driver drowsiness detection model, users can effectively monitor signs of driver fatigue and receive timely alerts. This significantly improves road safety and reduces the likelihood of accidents caused by drowsiness.

5.2 LIMITATION

Limitations of a study are its flaws and shortcomings that impacted or influenced the interpretation of the findings from the research. Firstly, there is the challenge of the dataset used for this study. The dataset, sourced from Kaggle, comprises images categorizing instances of open eyes and closed eyes, with a total of 2000 images for each category. The relatively small size of the dataset may limit the model's performance. Larger datasets with more diverse examples would likely enhance the algorithm's accuracy and robustness.

Next is the implementation of the data collection method. Due to limited experience in primary data collection, the study relied on pre-existing datasets from Kaggle rather than collecting real-time data directly. The datasets available on the platform may not fully represent the wide range of conditions encountered in real-world driving scenarios, such as varying lighting conditions and partial occlusions.

Additionally, the pre-existing datasets may not be updated frequently, impacting the relevance and applicability of the findings.

Furthermore, there are specific constraints and limitations associated with the use of the Haar Cascade Classifier algorithm for face detection. The accuracy of this algorithm depends on several factors, such as the quality and quantity of the data used for training, the complexity of the features it is looking for, and the adjustments made to its settings. While the Haar Cascade algorithm is generally effective at detecting faces in images and videos when properly trained and configured, it may not perform well in certain situations. For instance, it could struggle under challenging lighting conditions or when faces are partially covered. The algorithm tends to work better in environments with good lighting and clear images. It is important to be aware of these limitations and continuously evaluate how well the Haar Cascade Classifier algorithm performs under the specific conditions of its use.

5.3 FUTURE WORK

There are some limitations to this study, but there are also potential improvements for future research. As the field of driver drowsiness detection evolves, larger and more diverse datasets can be collected to train and test the system, enhancing its accuracy and robustness. In the future, expanding the dataset will allow for a more comprehensive evaluation of the Haar Cascade classifiers and other algorithms used for detecting driver drowsiness.

Future research should also explore the application of additional classifiers and algorithms. For instance, experimenting with Logistic Regression, Random Forests, Recurrent Neural Networks, Neural Networks, and K-Nearest Neighbours could provide valuable insights. While the Haar Cascade classifiers have been effective, no single algorithm is universally optimal. Comparing the performance of various algorithms will yield a more robust and versatile drowsiness detection system.

Furthermore, research could investigate the integration of the system with different hardware platforms and environments to ensure its effectiveness under diverse

real-world conditions. Additionally, exploring real-time data collection methods, such as directly capturing and analyzing live video feeds, would enhance the system's applicability and reliability.

By addressing these areas, future work aims to improve the accuracy and applicability of driver drowsiness detection systems, ultimately contributing to safer roads and reducing the incidence of accidents caused by driver fatigue.

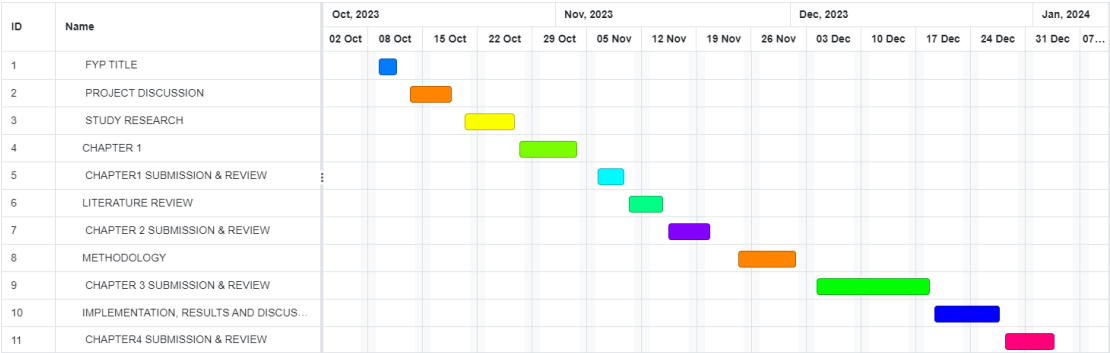
REFERENCES

1. Adochiei, I., Știrbu, O., Adochiei, N. I., Pericle-Gabriel, M., Larco, C., Mustață, Ș., & Costin, D. (2020). Drivers' drowsiness detection and warning systems for critical infrastructures. 2020 International Conference on e-Health and Bioengineering (EHB). <https://doi.org/10.1109/ehb50910.2020.9280165>
2. Ayachi, R., Afif, M., Said, Y., & Abdelali, A. B. (2021). Drivers Fatigue Detection Using EfficientDet In Advanced Driver Assistance Systems. IEEE. <https://doi.org/10.1109/ssd52085.2021.9429294>
3. P. Sudarshan, V. Bhardwaj and Virender, "Real-time Driver Drowsiness Detection and Assistance System using Machine Learning and IoT," 2023 8th International Conference on Communication and Electronics Systems (ICCES), Coimbatore, India, 2023, pp. 1128-1132, doi: 10.1109/ICCES57224.2023.10192831.
4. J. A. Marcial Basilio, G. Aguilar Torres, G. Sanchez Perez, L. K. Toscano Medina, H. M. Perez Meana, and E. E. Hernadez, "Explicit Content Image Detection," Signal Image Process. An Int. J., vol. 1, no.2, pp. 47–58, Dec. 2010, doi: 10.5121/sipij.2010.1205.
5. Al Redhaei, A., Albadawi, Y., Mohamed, S., & Alnoman, A. (2022, February 1). Realtime Driver Drowsiness Detection Using Machine Learning. IEEE Xplore. <https://doi.org/10.1109/ASET53988.2022.9734801>
6. Prasath, N., Sreemathy, J., & Vigneshwaran, P. (2022, March 1). Driver Drowsiness Detection Using Machine Learning Algorithm. IEEE Xplore. <https://doi.org/10.1109/ICACCS54159.2022.9785167>
7. Kamarudin, N., Jumadi, N. A., Mun, N. L., Keat, N. C., Ching, A. H. K., Mahmud, W. M. H. W., Morsin, M., & Mahmud, F. (2019). Implementation of Haar Cascade Classifier and Eye Aspect Ratio for Driver Drowsiness Detection Using Raspberry Pi. Universal Journal of Electrical and Electronic Engineering, 6(5B), 67–75. <https://doi.org/10.13189/ujeee.2019.061609>
8. Eryl Nanda Pratama, & Maki, A. (2022). Drowsiness Detection System for Masked Face Based on Deep Neural Network and Haar Cascade. <https://doi.org/10.1109/icoseit55604.2022.10029948>
9. <https://www.kaggle.com/datasets/prasadvpatil/mrl-dataset?resource=download>

10. Kamarudin, N., Jumadi, N. A., Mun, N. L., Keat, N. C., Ching, A. H. K., Mahmud, W. M. H. W., Morsin, M., & Mahmud, F. (2019). Implementation of 0 Cascade Classifier and Eye Aspect Ratio for Driver Drowsiness Detection Using Raspberry Pi. *Universal Journal of Electrical and Electronic Engineering*, 6(5B), 67–75. <https://doi.org/10.13189/ujeee.2019.061609>
11. Doifode, V. (2023). IoT-Based Driver Drowsiness Detection System. This project uses MPU6050 sensors to monitor head movements and alert the driver if drowsiness is detected. The system aims to reduce the significant percentage of road accidents caused by driver fatigue.
12. Gao, Z., Chen, X., Xu, J., & Zong, J. (2022). A Comprehensive Vision-Based Model for Commercial Truck Driver Fatigue Detection. This study developed a model combining facial key point detection and LSTM to monitor driver fatigue in monotonous driving environments
13. Zhao, Y., Xie, K., Zou, Z., et al. (2020). Intelligent recognition of fatigue and sleepiness based on InceptionV3-LSTM via multi-feature fusion. *IEEE Access*, 8, 144205–144217. This paper presents a method integrating InceptionV3 and LSTM for multi-feature fusion to detect driver fatigue
14. Nanda Pratama, E., & Maki, A. (2022). Drowsiness Detection System for Masked Face Based on Deep Neural Network and Haar Cascade. This research focuses on detecting drowsiness in drivers wearing masks, utilizing deep neural networks combined with the Haar Cascade algorithm
15. Sudarshan, P., Bhardwaj, V., & Virender. (2023). Real-time Driver Drowsiness Detection and Assistance System using Machine Learning and IoT. This study implements a real-time detection system using machine learning techniques and IoT to monitor driver alertness and provide timely warnings
16. Alioua, N., Amine, A., Rziza, M. (2020). Driver's fatigue detection based on yawning extraction. This paper describes a method for detecting driver fatigue by identifying yawning using computer vision techniques
17. Tibrewal, M., Srivastava, A., & Kayalvizhi, R. (2021). A deep learning approach to detect driver drowsiness. *International Journal of Engineering Research and Technology*, 10, 183–189. This research explores a deep learning framework for identifying signs of driver drowsiness using video and image analysis

APPENDIX A

GANTT CHART



APPENDIX B

EARLY WORK- INITIAL CODING

```
import cv2
import time
import os

face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier('haarcascade_eye_tree_eyeglasses.xml')

cap = cv2.VideoCapture(0)
a=0
b=0
c=time.time()
while 1:
    ret, img = cap.read()
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    if len(faces) == 0:
        print("No face detected")

    for (x,y,w,h) in faces:
        cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)

        roi_gray = gray[y:y+h, x:x+w]
        roi_color = img[y:y+h, x:x+w]

        eyes = eye_cascade.detectMultiScale(roi_gray)

        if (time.time()-c) >= 15:
            if a/(a+b)>=0.2:
                os.system('buzz.mp3')
                print("*****ALERT*****",a,b,a/(a+b))

            else:
                print("safe",a//((a+b))
                c=time.time()
                a=0
                b=0

        if len(eyes) == 0:
            a=a+1
            print('no eyes!!!')
        else:
            b=b+1
            print('eyes!!!')

        for (ex,ey,ew,eh) in eyes:
            cv2.rectangle(roi_color,(ex,ey),(ex+ew,ey+eh),(0,255,0),2)
    cv2.imshow('img',img)

    k = cv2.waitKey(1) & 0xff
    if k == 27:
        print("noeyes: ",a)
        print("total: ",(a+b))
        break

cap.release()
cv2.destroyAllWindows()
```

APPENDIX C

Trained model - Code

```
import os
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau

# Paths to your dataset
dataset_path = 'C:\\Users\\alyaf\\Desktop\\drowsiness-detection-
master\\Dataset'
train_folder = os.path.join(dataset_path, 'Train')
test_folder = os.path.join(dataset_path, 'Test')
categories = ['Open_Eyes', 'Closed_Eyes']

# Define image data generators with more extensive data augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
test_datagen = ImageDataGenerator(rescale=1./255)

# Load and preprocess training and validation images using
flow_from_directory
train_generator = train_datagen.flow_from_directory(
    train_folder,
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary',
    subset='training'
)

validation_generator = train_datagen.flow_from_directory(
    train_folder,
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary',
    subset='validation'
)

# Load and preprocess testing images using flow_from_directory
test_generator = test_datagen.flow_from_directory(
    test_folder,
    target_size=(224, 224),
```

```

        batch_size=32,
        class_mode='binary'
    )

# Model training
def build_and_train_model(train_generator, validation_generator):
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224,
3)),
        MaxPooling2D((2, 2)),
        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),
        Conv2D(128, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),
        Flatten(),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])

    model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

    early_stopping = EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True)
    reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2,
patience=5, min_lr=0.0001)

    model.fit(
        train_generator,
        epochs=50,
        validation_data=validation_generator,
        callbacks=[early_stopping, reduce_lr]
    )

    return model

model = build_and_train_model(train_generator, validation_generator)

# Model testing
def test_model(model, test_generator):
    y_pred = (model.predict(test_generator) > 0.5).astype("int32")
    y_true = test_generator.classes
    print("Classification Report:")
    print(classification_report(y_true, y_pred,
target_names=categories))
    print("Confusion Matrix:")
    print(confusion_matrix(y_true, y_pred))

test_model(model, test_generator)

# Save the model
def save_model(model):
    model.save('drowsiness_detection_model.h5')
    print("Model saved as 'drowsiness_detection_model.h5'")

save_model(model)

```

APPENDIX D

Real-Time Detection - Code

```
import cv2
import time
import os
import numpy as np
from tensorflow.keras.models import load_model

# Load the pre-trained model
model = load_model('drowsiness_detection_model.h5')

# Load the cascades
face_cascade =
cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
eye_cascade =
cv2.CascadeClassifier('haarcascade_eye_tree_eyeglasses.xml')

# Start video capture
cap = cv2.VideoCapture(0)
a = 0
b = 0
c = time.time()
no_eyes_counter = 0
no_eyes_threshold = 2 # Number of consecutive "no eyes" detections
before alert

def preprocess_eye(eye_img):
    eye_img = cv2.resize(eye_img, (224, 224))
    eye_img = eye_img.astype('float32') / 255.0
    eye_img = np.expand_dims(eye_img, axis=0)
    return eye_img

while True:
    ret, img = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    if len(faces) == 0:
        print("No face detected")

    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)

        roi_gray = gray[y:y + h, x:x + w]
        roi_color = img[y:y + h, x:x + w]

        eyes = eye_cascade.detectMultiScale(roi_gray)

        if (time.time() - c) >= 15:
            if a / (a + b) >= 0.2:
                os.system('buzz.mp3')
                print("****ALERT****", a, b, a / (a + b))
            else:
                print("safe", a / (a + b))
            c = time.time()
```

```

        a = 0
        b = 0

    if len(eyes) == 0:
        a += 1
        no_eyes_counter += 1
        print('no eyes!!!')
        if no_eyes_counter >= no_eyes_threshold:
            os.system('buzz.mp3')
            print("****ALERT: No eyes detected for three
consecutive frames****")
            no_eyes_counter = 0 # Reset the counter after alert
        else:
            no_eyes_counter = 0 # Reset the counter if eyes are
detected
        for (ex, ey, ew, eh) in eyes:
            eye_img = roi_color[ey:ey+eh, ex:ex+ew]
            eye_img_preprocessed = preprocess_eye(eye_img)
            prediction = model.predict(eye_img_preprocessed)
            if prediction < 0.5:
                a += 1
                print('Closed eyes detected')
            else:
                b += 1
                print('Open eyes detected')
            cv2.rectangle(roi_color, (ex, ey), (ex + ew, ey + eh),
(0, 255, 0), 2)

    cv2.imshow('img', img)

    k = cv2.waitKey(1) & 0xff
    if k == 27:
        print("noeyes: ", a)
        print("total: ", (a + b))
        break

cap.release()
cv2.destroyAllWindows()

```

